

Java Backend Architecture

Interview Series

Chapter 7.1

Maven Build Lifecycle & POM

50+ Interview Q&A

Code Examples

Architecture Diagrams

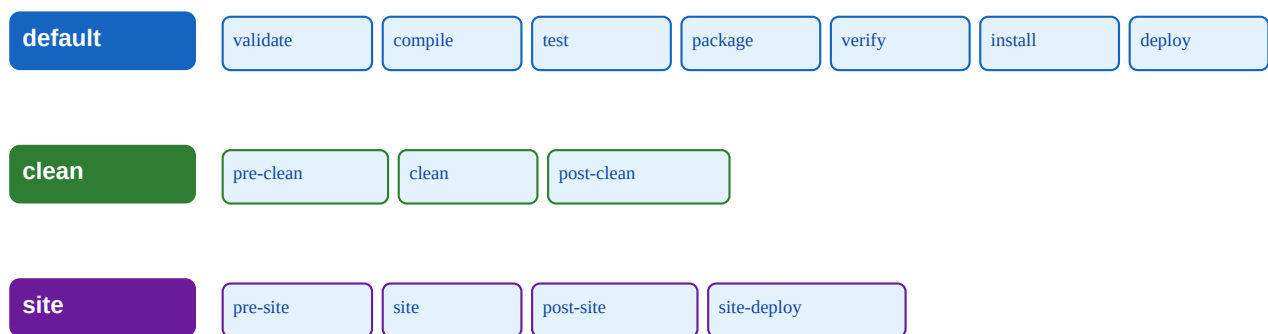
Advanced Topics

Chapter 7.1 — Maven Build Lifecycle & POM

Introduction

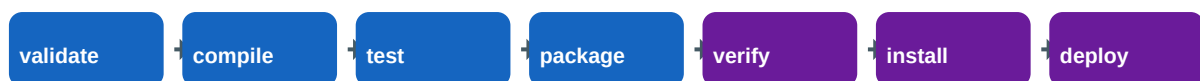
Apache Maven is a project management and comprehension tool based on the concept of a Project Object Model (POM). It standardises the build process across Java projects through three built-in lifecycles: **default**, **clean**, and **site**. Understanding these lifecycles and the POM structure is foundational to working with Maven effectively in enterprise environments.

Three Build Lifecycles



The **default** lifecycle handles project deployment through 23 phases (validate → compile → test → package → verify → install → deploy). The **clean** lifecycle removes files generated at build-time (pre-clean, clean, post-clean). The **site** lifecycle generates project documentation. Each phase triggers all preceding phases in its lifecycle.

Key Default Lifecycle Phases

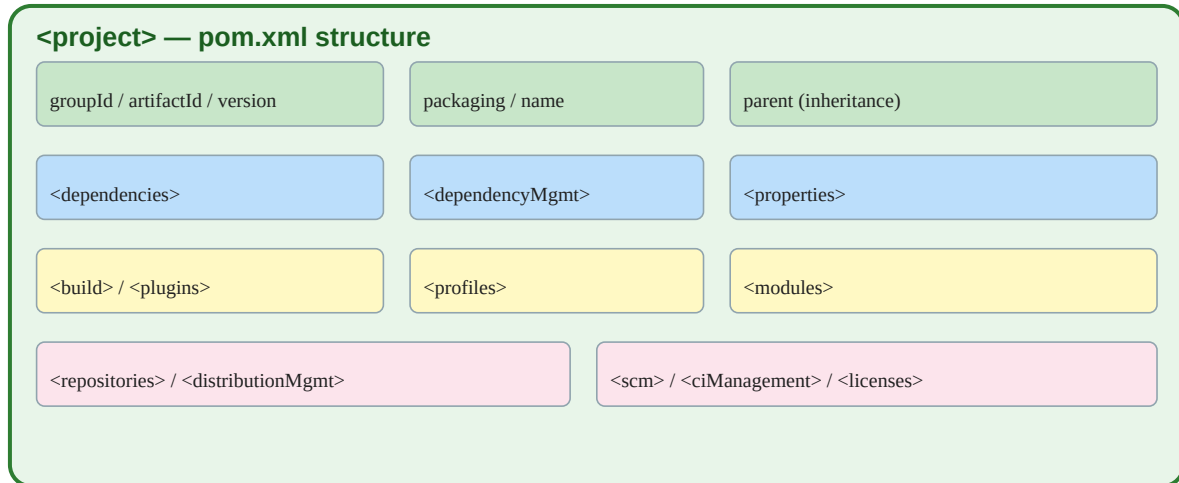


Each phase triggers all preceding phases — mvn package runs: validate→compile→test→package

Phase	Plugin:Goal (default binding)	Description
validate	—	Validate project structure
generate-sources	build-helper:generate-sources	Generate additional source code
compile	maven-compiler-plugin:compile	Compile main sources
test-compile	maven-compiler-plugin:testCompile	Compile test sources
test	maven-surefire-plugin:test	Run unit tests
package	maven-jar-plugin:jar	Package compiled code into JAR/WAR
verify	maven-failsafe-plugin:verify	Run integration tests
install	maven-install-plugin:install	Install to local ~/.m2/repository

deploy	maven-deploy-plugin:deploy	Upload to remote repository
--------	----------------------------	-----------------------------

POM Structure



The POM (Project Object Model) is Maven's fundamental unit of work — an XML file named **pom.xml**. Every project has exactly one POM. It contains project metadata, dependencies, build configuration, plugins, profiles, repositories, and more. Maven's "convention over configuration" philosophy means defaults apply unless explicitly overridden.

Minimal POM Example

```
<!-- minimal pom.xml --> <project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd"> <modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId> <artifactId>my-app</artifactId>
<version>1.0.0-SNAPSHOT</version> <packaging>jar</packaging> <properties>
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> </properties>
<dependencies> <dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId> <version>3.2.0</version> </dependency>
</dependencies> </project>
```

Effective POM

The **effective POM** is the merged result of your project's POM, all parent POMs, and the Super POM (Maven's built-in defaults). Run **mvn help:effective-pom** to inspect it. The Super POM defines default directories (`src/main/java`, `src/test/java`), default plugin versions, and Central repository. Understanding the effective POM helps debug unexpected build behaviour.

Maven Reactor & Build Flags

Flag	Full Option	Behaviour
-pl	--projects	Build only specified projects (comma-separated)

-am	--also-make	Build projects required by -pl targets
-amd	--also-make-dependents	Build projects that depend on -pl targets
-rf	--resume-from	Resume reactor build from specified project
-T	--threads	Parallel build (e.g., -T 4 or -T 1C)
-o	--offline	Work offline — use only local repository
-U	--update-snapshots	Force update of remote SNAPSHOT artifacts
-fae	--fail-at-end	Fail at end rather than immediately

Maven Wrapper (mvnw)

The Maven Wrapper (mvnw) ensures that all team members and CI pipelines use the same Maven version. Generate it with: **mvn wrapper:wrapper -Dmaven=3.9.6**. This creates mvnw, mvnw.cmd, and .mvn/wrapper/maven-wrapper.properties. Commit these files to source control so any developer can run ./mvnw without installing Maven.

```
# Generate Maven Wrapper mvn wrapper:wrapper -Dmaven=3.9.6 # Use wrapper ./mvnw clean install #
.mvn/wrapper/maven-wrapper.properties distributionUrl=https://repo.maven.apache.org/maven2/org
/apache/maven/apache-maven/3.9.6/apache-maven-3.9.6-bin.zip wrapperUrl=https://repo.maven.apac
he.org/maven2/org/apache/maven/wrapper/maven-wrapper/3.2.0/maven-wrapper-3.2.0.jar
```

CI-Friendly Versions

CI-friendly versions allow the build version to be set from the command line or CI environment variables, enabling clean pipelines without modifying pom.xml. Supported since Maven 3.5.

```
<!-- pom.xml --> <version>${revision}${changelist}</version> <properties>
<revision>1.0.0</revision> <changelist>-SNAPSHOT</changelist> </properties> <!-- Must add
flatten-maven-plugin to resolve ${revision} in installed/deployed POM --> <build> <plugins>
<plugin> <groupId>org.codehaus.mojo</groupId> <artifactId>flatten-maven-plugin</artifactId>
<version>1.6.0</version> <configuration><updatePomFile>true</updatePomFile></configuration>
<executions> <execution> <id>flatten</id> <phase>process-resources</phase>
<goals><goal>flatten</goal></goals> </execution> </executions> </plugin> </plugins> </build> #
Override at build time ./mvnw clean deploy -Drevision=2.1.0 -Dchangelist=
```

Common mvn Commands

Command	Purpose
mvn clean install	Full build + install to local repo
mvn clean install -DskipTests	Skip unit test execution (still compiles tests)
mvn clean install -Dmaven.test.skip=true	Skip compilation AND execution of tests
mvn dependency:tree	Print full dependency tree
mvn dependency:analyze	Find unused declared / undeclared used deps
mvn versions:display-dependency-updates	Show available dependency upgrades
mvn help:effective-pom	Show merged effective POM
mvn help:effective-settings	Show merged effective settings

<code>mvn help:describe -Dplugin=compiler</code>	Describe plugin goals and parameters
--	--------------------------------------

50+ Interview Questions & Answers

Q1. What are Maven's three built-in lifecycles?

default (build/deploy), clean (remove build output), site (generate documentation).

Q2. How many phases does the default lifecycle have?

23 phases, from validate to deploy.

Q3. What is the Super POM?

Maven's built-in default POM that all project POMs implicitly inherit from. It defines default directory structure, Central repository, and default plugin versions.

Q4. What is the effective POM?

The merged result of your POM + all parent POMs + Super POM. View it with: `mvn help:effective-pom`.

Q5. What does `mvn package` do?

Runs all phases up to and including package (validate→compile→test→package), producing a JAR/WAR.

Q6. What is the difference between `mvn install` and `mvn deploy`?

`install` puts the artifact in the local `~/.m2` repository; `deploy` uploads it to a remote repository (Nexus/Artifactory).

Q7. What is the default source directory in Maven?

`src/main/java` (main code), `src/test/java` (test code), `src/main/resources` (resources).

Q8. How do you skip tests in Maven?

`-DskipTests` skips execution but compiles tests; `-Dmaven.test.skip=true` skips both compilation and execution.

Q9. What is the purpose of the `groupId`, `artifactId`, and `version` (GAV) coordinates?

They uniquely identify an artifact in a Maven repository. `groupId` is the organisation/package, `artifactId` is the component name, `version` is the release version.

Q10. What packaging types does Maven support?

`jar` (default), `war`, `ear`, `pom` (parent/aggregator), `maven-plugin`, `aar` (Android). Plugins can add custom types.

Q11. What is a SNAPSHOT version?

A version ending in `-SNAPSHOT` means the artifact is under active development and may change. Maven re-downloads SNAPSHOTs (within a configurable interval) from repositories.

Q12. What is the Maven local repository?

`~/.m2/repository` — a local cache where Maven stores all downloaded artifacts and installed project artifacts.

Q13. How do you configure the Maven compiler version?

Set `maven.compiler.source` and `maven.compiler.target` properties, or configure `maven-compiler-plugin` with `source` or `target` parameter.

Q14. What is the parameter in `maven-compiler-plugin`?

Introduced in Java 9, `release` sets source, target, and the bootstrap class path in one step, ensuring correct cross-compilation.

Q15. What does mvn clean do?

Runs the clean lifecycle: deletes the target/ directory (pre-clean, clean, post-clean phases).

Q16. What is the difference between -DskipTests and -Dmaven.test.skip?

-DskipTests compiles tests but skips execution. -Dmaven.test.skip=true skips both compilation and execution. The former is faster when tests were already compiled.

Q17. What is a Maven profile?

A set of configuration overrides activated conditionally — by -P flag, environment variable, OS, JDK version, or file presence. Used for environment-specific builds.

Q18. How do you activate a Maven profile?

mvn clean install -Pproduction. Also via , environment variable, JDK version, OS, or file presence/absence conditions.

Q19. What is the Maven Reactor?

The component that builds multi-module projects. It determines build order from inter-module dependencies and supports -pl, -am, -amd, -rf flags.

Q20. What does -pl flag do?

Limits the reactor build to specified projects: mvn -pl module-a,module-b clean install.

Q21. What does -am flag do?

Also Make — builds all projects that the -pl targets depend on, ensuring their dependencies are up to date.

Q22. What does -amd flag do?

Also Make Dependents — builds all projects that depend on the -pl targets, useful for impact analysis.

Q23. What is the Maven Wrapper (mvnw)?

A shell script that downloads and uses a specific Maven version, ensuring build reproducibility across environments without requiring Maven installation.

Q24. What are CI-friendly versions in Maven?

Using \${revision}, \${sha1}, \${changelist} placeholders in to allow version injection from command line or CI. Requires flatten-maven-plugin for published artifacts.

Q25. What is the flatten-maven-plugin used for?

Resolves CI-friendly version placeholders (\${revision}, etc.) in the POM before installing/deploying, so published artifacts have literal version strings.

Q26. What does mvn dependency:tree show?

The full transitive dependency tree, showing how each dependency was pulled in and at what version.

Q27. How do you exclude a transitive dependency?

Use inside the direct :

Q28. What is Maven's nearest-wins strategy?

When multiple versions of the same dependency exist in the dependency tree, Maven selects the version nearest to the root (fewest hops). Explicit declarations always win.

Q29. What does mvn versions:display-dependency-updates show?

Lists all dependencies that have newer versions available in the configured repositories.

Q30. What is the difference between compile and provided scope?

compile is available at compile/test/runtime time and packaged in WAR. provided is available at compile/test time but not packaged — provided by the runtime container (e.g., servlet-api on Tomcat).

Q31. What is the test dependency scope?

Available only for test compilation and execution. Not included in the final artifact or its transitive dependencies.

Q32. What is the import scope?

Used with pom packaging in to import a BOM (Bill of Materials), merging the BOM's managed dependency versions into the current project.

Q33. What is a BOM (Bill of Materials)?

A POM with packaging=pom containing only entries. Imported into projects to manage dependency versions centrally (e.g., Spring Boot BOM).

Q34. How does parent POM inheritance work?

Child POMs inherit groupId/version (if not overridden), dependencies, dependencyManagement, build plugins, properties, and profiles from parent. Declare with element.

Q35. What is the difference between and ?

adds dependencies immediately. only sets version/scope/exclusion defaults — child projects must still declare the dependency to use it.

Q36. What is the pluginManagement section?

Analogous to dependencyManagement — configures plugin versions and configuration centrally in parent POM; child modules must still declare the plugin to activate it.

Q37. How do you bind a plugin goal to a lifecycle phase?

In the plugin's element: packageshade.

Q38. What is the maven-surefire-plugin?

Runs unit tests during the test phase. Supports JUnit 4/5, TestNG. Can be configured to include/exclude test patterns, set system properties, and configure parallel execution.

Q39. What is the maven-failsafe-plugin?

Runs integration tests during verify phase (pre-integration-test, integration-test, post-integration-test, verify). Designed to continue through test failures to allow teardown.

Q40. What does mvn help:describe -Dplugin=compiler show?

Lists all goals of the maven-compiler-plugin with their parameters, default values, and descriptions.

Q41. What is the archetype plugin used for?

Generates project skeletons from templates: mvn archetype:generate -DarchetypeGroupId=... Creates standard project structures.

Q42. What is the settings.xml file?

Maven's user-level configuration file (~/.m2/settings.xml) and optional global file (\${M2_HOME}/conf/settings.xml). Configures mirrors, servers (credentials), proxies, and active profiles.

Q43. How do you configure repository credentials in Maven?

In settings.xml section: nexususerpass. The server id must match the repository id in pom.xml.

Q44. What is a mirror in Maven settings?

A mirror redirects requests for a repository to another URL. Typically used to route all artifact requests through an internal repository manager (Nexus/Artifactory).

Q45. What is offline mode in Maven?

`mvn -o / --offline` — Maven uses only locally cached artifacts. Fails if any required artifact isn't already in `~/.m2`.

Q46. How does Maven determine dependency version when two versions appear at the same depth?

When two versions appear at equal depth, Maven uses the first-declared wins strategy (order in `pom.xml` determines selection).

Q47. What is the maven-enforcer-plugin?

Enforces project rules such as minimum Maven/JDK version, banned dependencies, no SNAPSHOT dependencies in releases, and custom rules.

Q48. How do you create a Maven release?

Use `maven-release-plugin`: `mvn release:prepare` (tags SCM, bumps version) then `mvn release:perform` (checks out tag, builds, deploys). Requires SCM configuration.

Q49. What does `mvn dependency:analyze` report?

Identifies: (1) declared but unused dependencies, and (2) used but undeclared dependencies (accessed transitively). Helps keep `pom.xml` accurate.

Q50. What is the `.mvn` directory used for?

Contains Maven Wrapper files (`wrapper/maven-wrapper.properties`), project-level JVM config (`jvm.config` — e.g., `-Xmx2g`), and Maven config (`maven.config` — e.g., `-T 4 --batch-mode`).

Q51. What is `--batch-mode` in Maven?

`mvn --batch-mode` (or `-B`) suppresses interactive prompts and disables progress output — essential for CI pipelines.

Q52. What is the difference between `mvn verify` and `mvn install`?

`verify` runs through the `verify` phase (including integration tests) but does NOT install to local repo. `install` runs through `install` phase, copying the artifact to `~/.m2`.

Q53. How do you run a single test class with Maven?

`mvn test -Dtest=MyServiceTest` or `mvn test -Dtest=MyServiceTest#myMethod`. Uses `maven-surefire-plugin`.

Q54. What is the `maven-shade-plugin`?

Creates an uber-jar (fat JAR) containing the project classes and all dependencies. Can relocate packages to avoid classpath conflicts.

Q55. What does `in` in a profile do?

Defines conditions under which the profile is automatically activated: by JDK version, OS, environment variable, system property, or file presence/absence.

Q56. How do you pass system properties to Maven builds?

`mvn clean test -DmyProperty=value`. Accessible in `pom.xml` as `${myProperty}` and in tests via `System.getProperty('myProperty')`.